

時空之門 The Ages
軟體架構文件 (Software Architecture Document)

版本 1.0

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

修訂紀錄 (Revision History)

日期	版本	說明	作者
2026/10/06	1.0	初版：依 main 分支（commit 42fc4b2）撰寫，涵蓋 4+1 架構視圖	Nepson Lee

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

目錄 (Table of Contents)

1. 簡介 (Introduction)	5
1.1 目的 (Purpose)	5
1.2 範圍 (Scope)	5
1.3 定義、縮寫與名詞 (Definitions, Acronyms, and Abbreviations)	5
1.4 參考文件 (References)	5
2. 架構表示法 (Architectural Representation)	6
3. 架構目標與限制 (Architectural Goals and Constraints)	6
4. 使用案例視圖 (Use-Case View)	6
4.1 架構關鍵使用案例說明 (Significant Use Case Descriptions)	7
4.1.1 登入並進入遊戲世界	7
4.1.2 戰鬥與擊殺	8
4.1.3 區域傳送	8
5. 邏輯視圖 (Logical View)	8
5.1 架構概觀 (Architecture Overview)	8
5.1.1 展示層 (Presentation Layer)	9
5.1.2 閘道層 (Gateway Layer)	9
5.1.3 遊戲核心層 (World Layer)	9
5.1.4 持久層 (Persistence Layer)	9
5.2 架構關鍵使用案例實現 (Architecturally Significant Use-Case Realizations)	9
5.2.1 登入並進入遊戲世界	10
5.2.2 戰鬥與擊殺	12
5.2.3 區域傳送	13
5.3 架構關鍵設計套件 (Architecturally Significant Design Packages)	13
5.3.1 區域事件迴圈 (Zone Tick)	15
5.3.2 通訊協定 (Protocol)	15
5.3.3 遊戲內容與驗證 (Content)	15
5.4 資料庫綱要 (Database Schema)	17
6. 程序視圖 (Process View)	17
6.1 瀏覽器主執行緒	18
6.2 Tomcat 請求執行緒	18
6.3 區域執行緒	18
6.4 存檔執行緒	18
6.5 共用狀態：PartyService	19
6.6 執行緒之間的規則	19
7. 部署視圖 (Deployment View)	19
7.1 用戶端 (瀏覽器)	21
7.2 靜態檔案伺服器／CDN	21

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

7.3	反向代理	21
7.4	遊戲伺服器	22
7.5	資料庫伺服器	22
7.6	快取伺服器（預留）	22
7.7	開發環境	22
8.	實作視圖（Implementation View）	22
8.1	概觀（Overview）	23
8.2	層級（Layers）	24
8.3	部署元件（Deployment Components）	24
8.4	建置與測試	24
9.	規模與效能（Size and Performance）	24
10.	品質（Quality）	24
10.1	安全性	24
10.2	可靠性	25
10.3	可維護性	25
10.4	已知限制	25

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

軟體架構文件 (Software Architecture Document)

1. 簡介 (Introduction)

1.1 目的 (Purpose)

本文件以多個架構視圖描述「時空之門 The Ages」系統的整體架構，呈現系統在不同面向的設計，並記錄已經做出的重要架構決策，作為開發、審查與後續擴充的共同依據。

1.2 範圍 (Scope)

「時空之門 The Ages」是向台灣經典 MUD《時空之門》（玩家暱稱 4444）致敬的瀏覽器 3D 多人線上遊戲。本文件涵蓋 GitHub 儲存庫 nepson-lee/the-ages 的 main 分支（commit 42fc4b2）所實作的系統：瀏覽器用戶端、遊戲伺服器、前後端通訊協定與資料庫。

遊戲內容的企劃細節（區域、NPC、物品、商店、任務、技能的數值）不在本文件範圍內，請參考 content.yml 與 README.md。

1.3 定義、縮寫與名詞 (Definitions, Acronyms, and Abbreviations)

縮寫／名詞	說明
MUD	Multi-User Dungeon，多人文字線上遊戲
SAD	Software Architecture Document，軟體架構文件
RUP	Rational Unified Process
SPA	Single-Page Application，單頁式網頁應用程式
區域 (Zone)	一張地圖，以及其中的玩家、NPC、地上物品與出口；每個區域由一條專屬執行緒推進
Tick	區域以固定頻率執行的一次遊戲邏輯更新（預設每秒 10 次）
NPC	Non-Player Character，由伺服器控制的角色，包括怪物、商人與任務 NPC
HP／MP	生命／內力（施放技能消耗內力）
JWT	JSON Web Token，登入後取得的存取權杖
Protobuf	Protocol Buffers，前後端共用的二進位訊息格式
WebSocket	瀏覽器與伺服器之間的雙向持久連線
權杖桶 (Token Bucket)	頻率限制演算法：允許短時間連續操作，但限制長時間的平均速度
Flyway	資料庫結構的版本管理工具

1.4 參考文件 (References)

1. README.md：專案說明、遊戲規則、設定與已知限制
2. protocol/src/main/protobuf/theages/v1/game.proto：前後端通訊協定
3. server/src/main/resources/world/content.yml：遊戲內容（區域、NPC、物品、商店、任務、技能）

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

4. server/src/main/resources/application.yml：伺服器設定
5. server/src/main/resources/db/migration (V1~V6)：資料庫結構
6. SAD_Sample.doc：本文件的格式範本

2. 架構表示法 (Architectural Representation)

本文件以一系列視圖呈現系統架構：使用案例視圖、邏輯視圖、程序視圖、部署視圖與實作視圖（4+1 視圖），並以「規模與效能」、「品質」兩章說明架構如何滿足非功能需求。

圖表使用統一塑模語言（UML）繪製，以 PlantUML 產生。圖表與內文中的類別、套件、設定名稱沿用程式碼中的英文原名，方便對照原始碼。

3. 架構目標與限制 (Architectural Goals and Constraints)

以下需求與限制對系統架構有重大影響：

- 以瀏覽器為唯一用戶端，玩家不需安裝任何程式；瀏覽器需支援 WebGL 與 WebSocket。
- 伺服器擁有最終決定權（server-authoritative）：客戶端只送出「意圖」（移動目標、攻擊對象、文字指令），位置、戰鬥、掉寶、交易等結果一律由伺服器計算，以防止作弊。
- 沿用 MUD 的 tick 制：戰鬥每秒一回合，不需要動作遊戲等級的即時同步，降低網路與伺服器負擔。
- 每個區域由單一執行緒推進：區域狀態只在自己的 tick 執行緒上讀寫，不需要鎖；其他執行緒只能把事件排入區域的佇列。
- tick 中禁止阻塞 I/O：角色存檔交給專用執行緒非同步寫入資料庫。
- 協定只有一份來源：game.proto 同時產生伺服器的 Java 程式碼與前端的 TypeScript 程式碼，前後端的訊息格式不會不一致。
- 遊戲內容由資料驅動：區域、NPC、物品、商店、任務與技能都寫在 content.yml；伺服器啟動時驗證所有引用，有錯誤就拒絕啟動。
- 安全：JWT 驗證、bcrypt 密碼雜湊、WebSocket 來源檢查、遊戲訊息與登入的頻率限制、帳號鎖定。
- 技術選型：伺服器為 Java 17、Spring Boot 4.1、Maven；資料庫為 PostgreSQL；前端為 TypeScript、Vite、Three.js。
- 目前以單台遊戲伺服器運作；架構保留往多節點擴充的空間（Redis 已列入部署規劃但尚未使用）。

4. 使用案例視圖 (Use-Case View)

使用案例視圖描述系統中具有架構意義的使用案例：它們代表系統的核心功能，或是經過大部分架構元素，或是凸顯架構中特別需要注意的設計。

系統的參與者有三種：玩家；隊友（其他玩家，參與組隊與聊天）；以及遊戲世界本身（區域 tick 驅動的 NPC 行為）。本系統有三個架構關鍵使用案例：

- 登入並進入遊戲世界
- 戰鬥與擊殺
- 區域傳送

下圖為系統的使用案例全貌，以底色標示的為架構關鍵使用案例。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

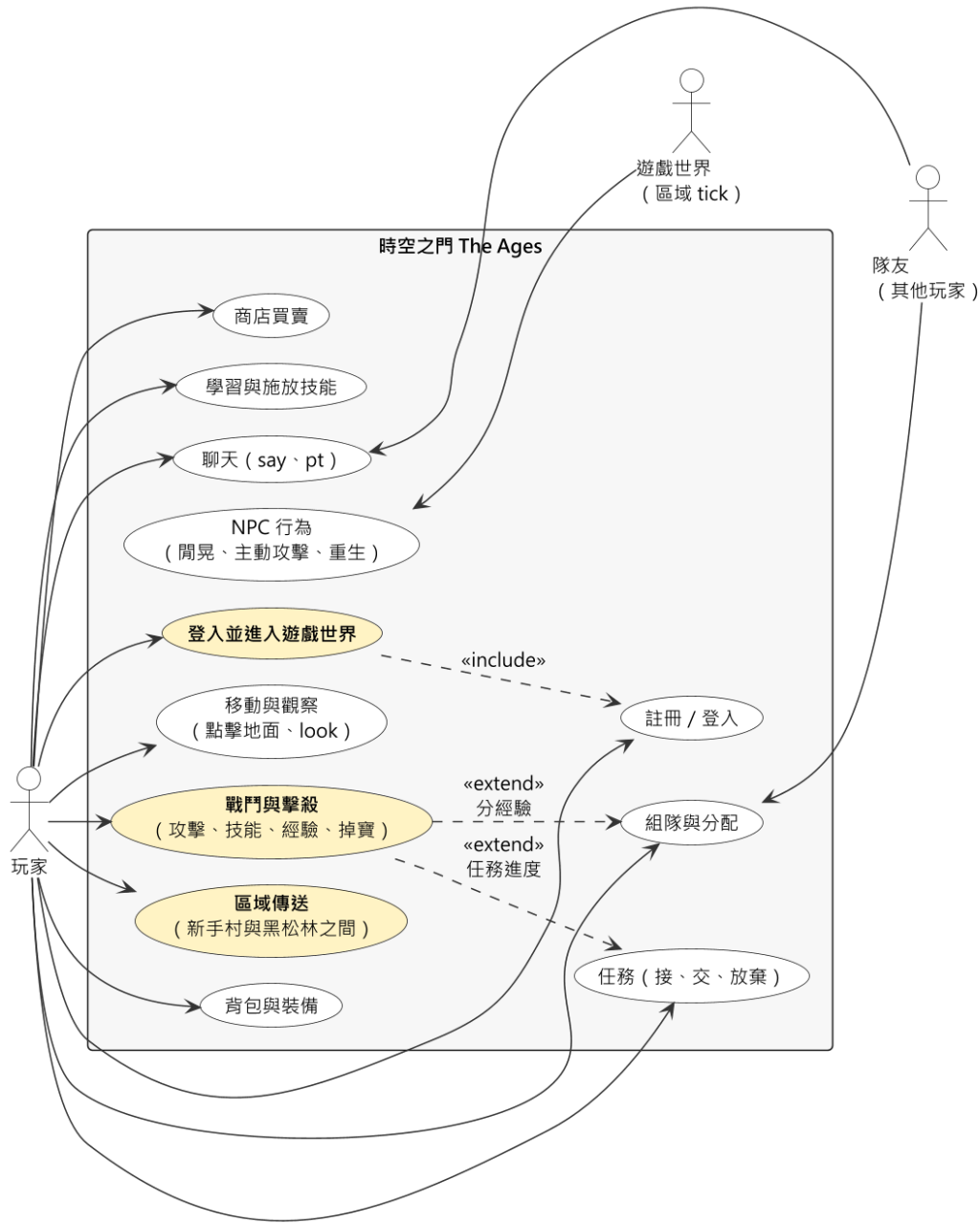


圖 1 使用案例圖

4.1 架構關鍵使用案例說明 (Significant Use Case Descriptions)

4.1.1 登入並進入遊戲世界

玩家在登入畫面輸入名字與密碼，伺服器驗證後發給 JWT。瀏覽器以 JWT 建立 WebSocket 連線，伺服器載入角色的等級、位置、銅錢、背包、任務與技能，將角色排入他所在的區域；區域在下一個 tick 建立玩家實體，送出歡迎訊息、完整場景快照與角色的各項狀態，瀏覽器據此建立 3D 場景。此使用案例經過登入頻率限制、帳號驗證、JWT、WebSocket 握手、資料庫載入、區域事件佇列，以及同

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

一角色重複登入的處理。

4.1.2 戰鬥與擊殺

玩家點擊 NPC 或輸入 kill 指令發起攻擊，也可以施放技能。角色先追到攻擊距離內，之後每秒一回合，由伺服器判定命中與傷害，NPC 會反擊；主動攻擊型的 NPC 會在玩家接近時先動手。

NPC 死亡時，伺服器在同一個 tick 內完成所有後續處理：依組隊規則分配經驗與銅錢、累計任務的擊殺進度、依掉寶表擲骰並在地上產生戰利品，並安排 NPC 在數秒後重生。此使用案例經過戰鬥、NPC 行為、組隊、任務、物品與技能等大部分遊戲核心元件。

4.1.3 區域傳送

玩家點擊出口光柱或輸入方向指令（例如 north），角色走到出口後移往另一個區域（新手村與黑松林之間）。由於每個區域各由一條執行緒推進，這個使用案例是唯一需要在兩個區域執行緒之間交接玩家狀態的流程，也牽涉到連線的訊息路由切換與換區途中斷線的處理。

5. 邏輯視圖 (Logical View)

邏輯視圖描述系統最重要的類別、它們如何組成套件與子系統，以及這些套件如何分層。系統分為四層：展示層（瀏覽器）、閘道層、遊戲核心層與持久層。

5.1 架構概觀 (Architecture Overview)

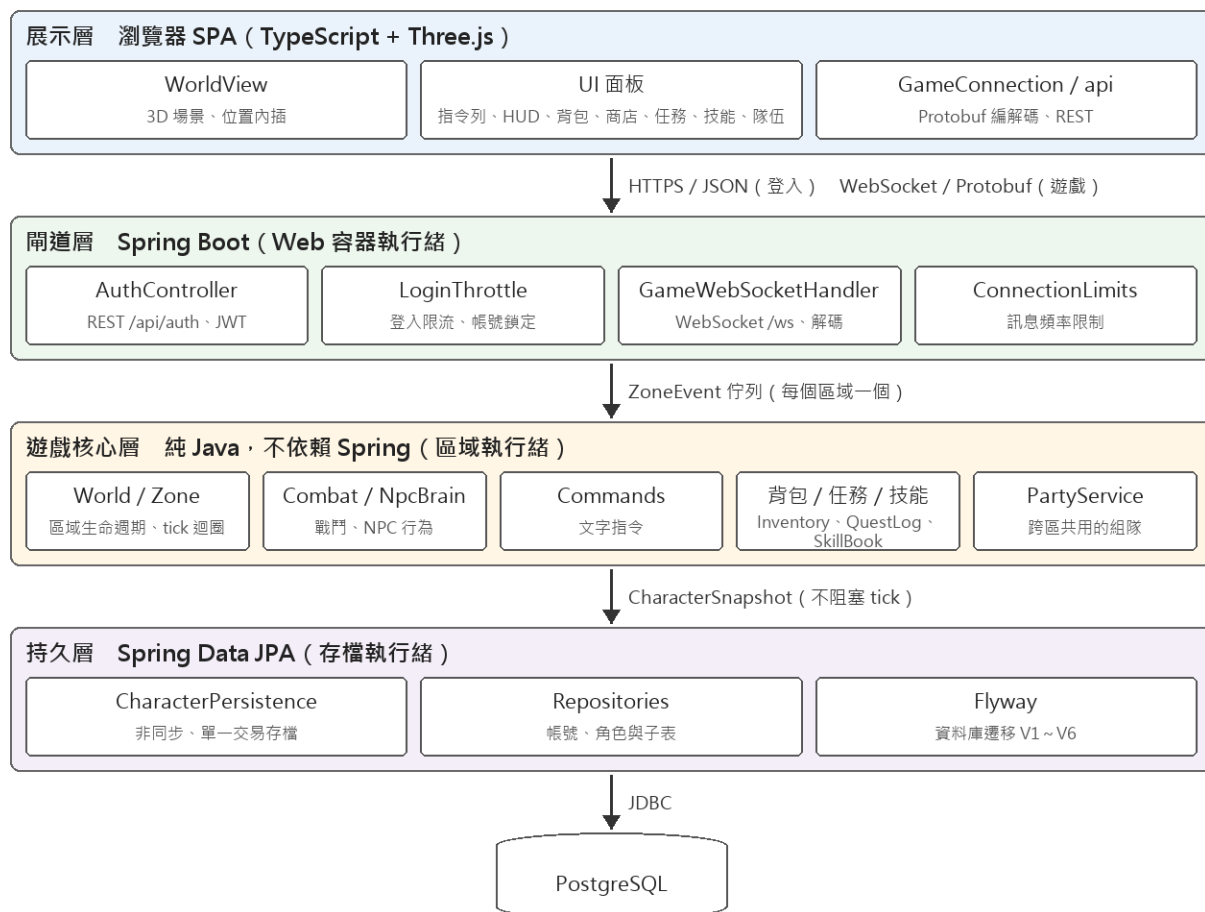


圖 2 分層架構

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

5.1.1 展示層 (Presentation Layer)

展示層是瀏覽器中執行的單頁式應用程式 (TypeScript + Three.js)，只負責顯示伺服器送來的狀態、收集玩家的動作，不做任何遊戲判定。

- game：3D 場景 (WorldView)、角色與物品的模型 (models)、各區域的地形與氣氛 (zones)。伺服器每秒 10 次的位置更新，在這裡以指數平滑補成每秒約 60 格的動畫。
- net：REST 登入 (api) 與 WebSocket 連線 (GameConnection)，負責 Protobuf 的編碼與解碼。
- ui：指令列、狀態列、背包、商店、任務、技能與隊伍面板。面板上的按鈕送出的就是文字指令，與玩家自己輸入的指令走同一條路徑。

5.1.2 閘道層 (Gateway Layer)

閘道層是伺服器的進出口，執行在 Web 容器的執行緒上，只負責驗證、解碼與限流，不碰觸任何遊戲狀態。

- AuthController / AuthService / TokenService：註冊、登入與 JWT 簽發；LoginThrottle 負責登入頻率限制與帳號鎖定。
- JwtHandshakeInterceptor：在 WebSocket 握手時驗證網址中的 JWT。
- GameWebSocketHandler：解碼 Protobuf 訊息、經過 ConnectionLimits 的頻率限制，再把訊息轉成 ZoneEvent，排入玩家目前所在的區域。
- WebSocketPlayerConnection：把 WebSocket 連線包裝成區域可用的 PlayerConnection，記住玩家目前所在的區域，送出訊息時不阻塞。

5.1.3 遊戲核心層 (World Layer)

遊戲核心層是純 Java 程式，不依賴 Spring，因此可以完全以單元測試驗證。所有遊戲規則都在這一層。

- World：建立並管理所有區域的生命週期。
- Zone：一個區域的 tick 迴圈與全部狀態 (玩家、NPC、地上物品、出口)。
- Combat、NpcBrain、Commands：戰鬥判定、NPC 行為與文字指令。
- world.item、world.quest、world.skill：背包與商店、任務日誌、技能與增益效果。
- world.party：PartyService，唯一跨區域共用的遊戲狀態。

5.1.4 持久層 (Persistence Layer)

- CharacterPersistence：在專用執行緒上把角色快照寫回資料庫；角色、背包、任務、技能在同一個交易內寫入。
- Spring Data JPA Repositories：帳號、角色與角色子表的存取。
- Flyway：伺服器啟動時套用資料庫遷移 (V1~V6)。

5.2 架構關鍵使用案例實現 (Architecturally Significant Use-Case Realizations)

本節以循序圖呈現三個架構關鍵使用案例如何由各層的元件協作完成。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

5.2.1 登入並進入遊戲世界

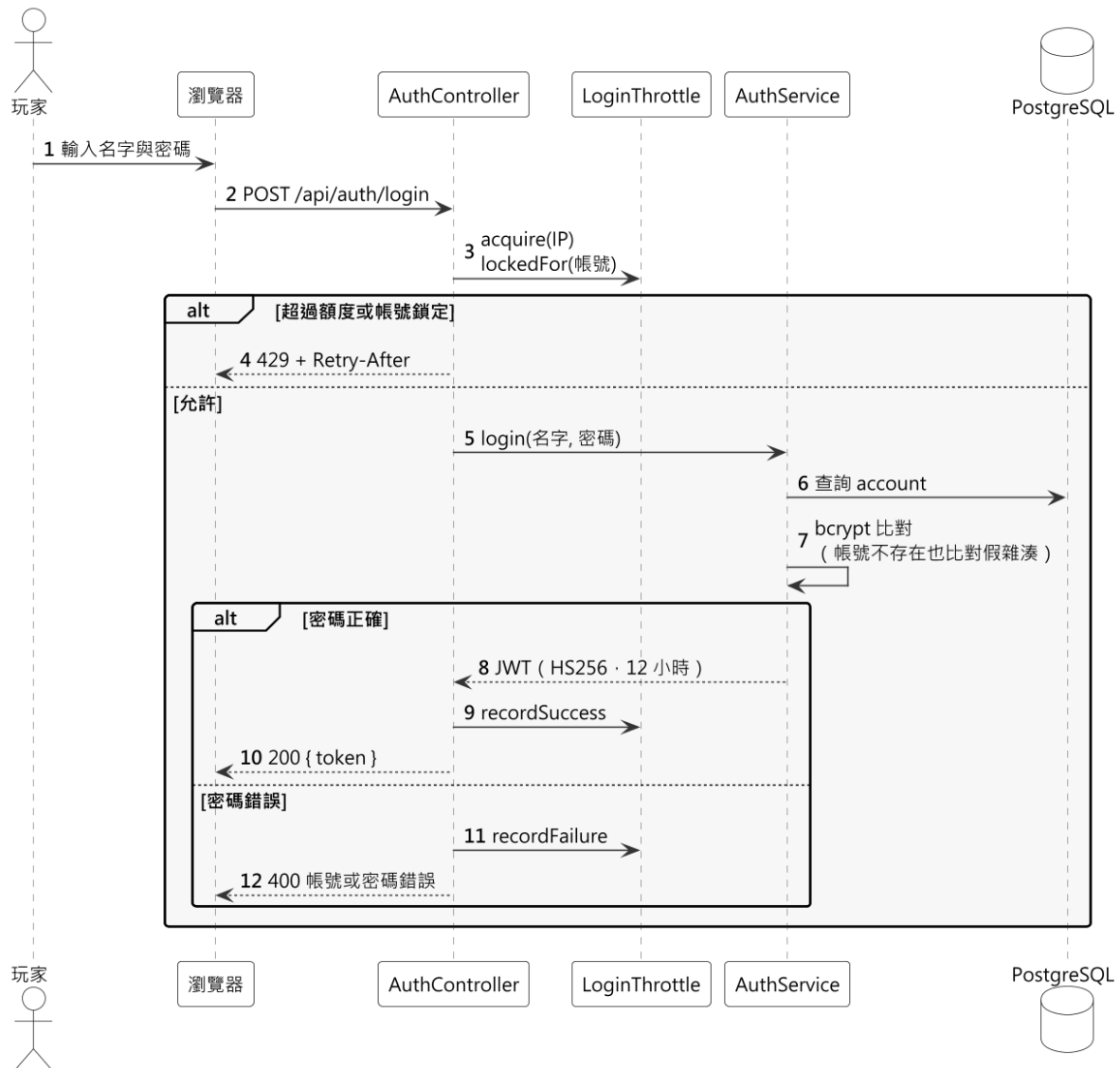


圖 3 登入 (REST)

- 登入時先檢查 IP 額度與帳號是否被鎖定；帳號鎖定期間連密碼都不檢查。
- 帳號不存在時也會做一次假的密碼雜湊比對，回應時間與帳號存在時一致，無法藉此試探帳號。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

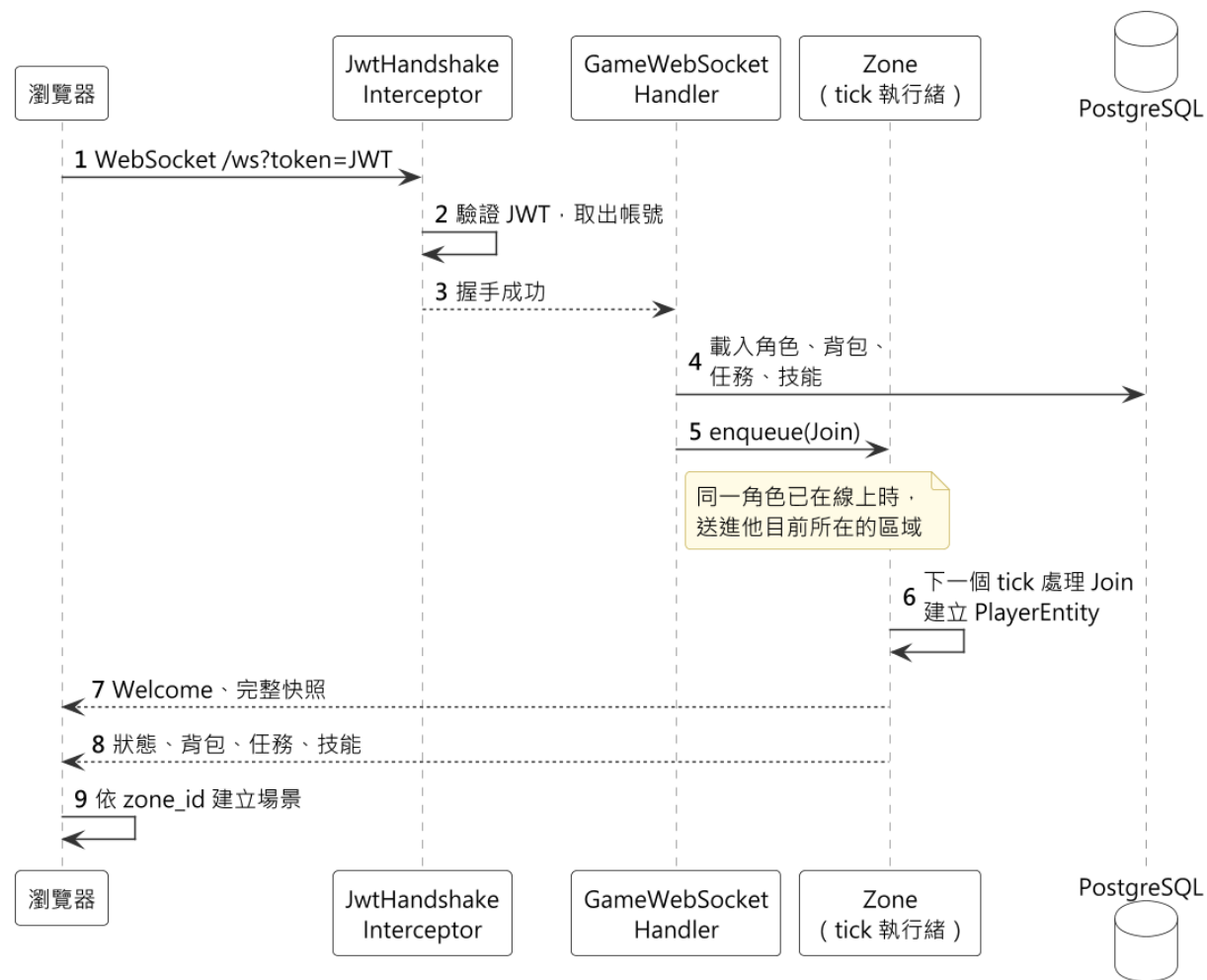


圖 4 進入遊戲世界（WebSocket）

- 同一角色已經在線上時，新連線會被送進他目前所在的區域，由區域沿用記憶體中的狀態並踢掉舊連線；資料庫中的位置可能尚未寫入，不能用來決定區域。

5.2.2 戰鬥與擊殺

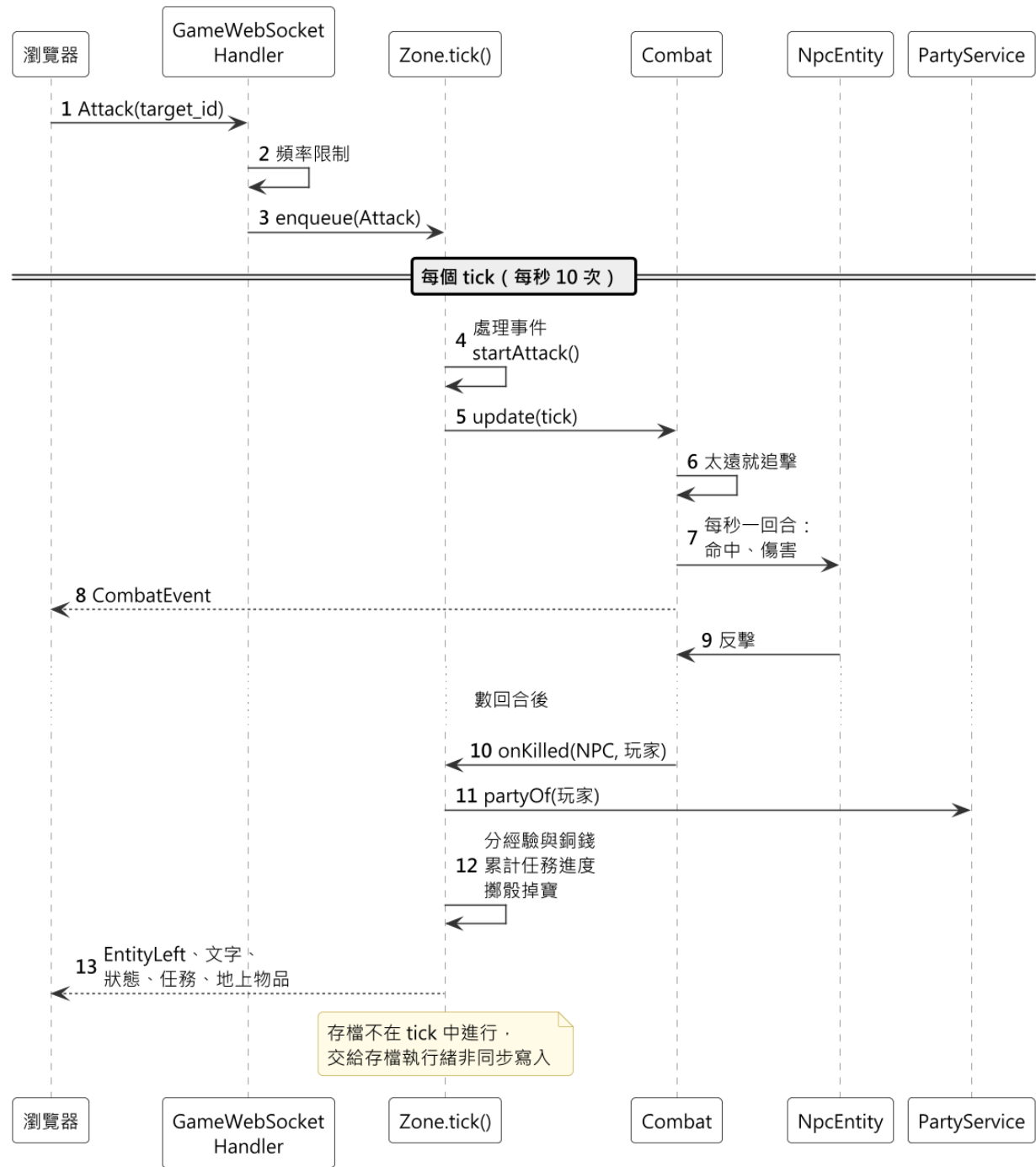


圖 5 戰鬥與擊殺

- 攻擊、技能與指令都只是排入佇列的事件，實際判定在區域的 tick 中進行。
- 技能攻擊與一般攻擊共用「打中之後」的處理（死亡、反擊），所以技能擊殺同樣會分配經驗、累計任務進度與掉寶。
- 分配對象為擊殺者加上同一區域、仍然活著、離屍體 30 公尺內的隊友。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

5.2.3 區域傳送

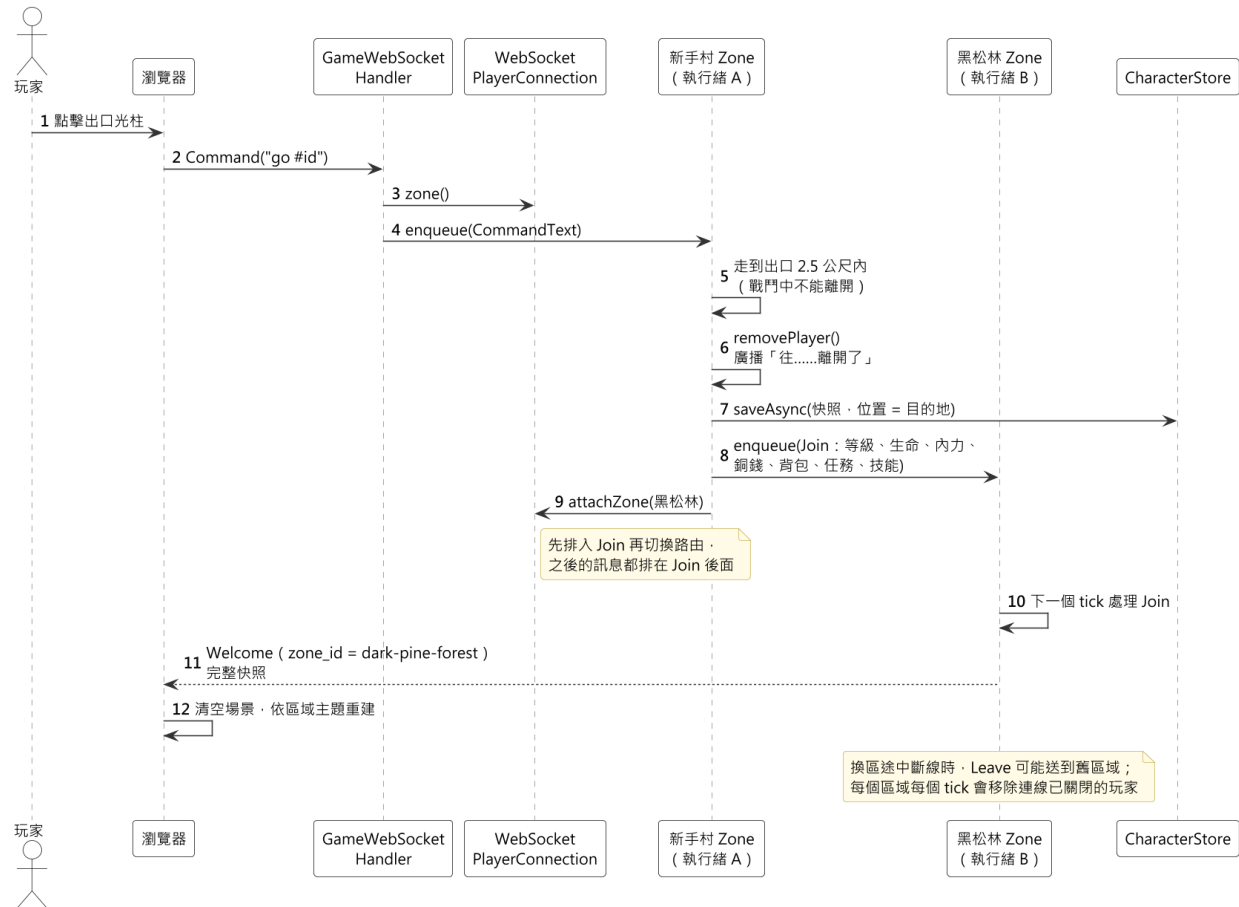


圖 6 區域傳送

- 原區域只透過目標區域的佇列交接玩家狀態（等級、生命、內力、銅錢、背包、任務、技能），不直接存取對方的資料。
- 先把 Join 排入目標區域，再切換連線的路由；之後收到的訊息都會排在 Join 之後。
- 換區當下立即存檔，位置為目的地；伺服器若在途中停止，玩家會出現在目的地。

5.3 架構關鍵設計套件 (Architecturally Significant Design Packages)

下圖為伺服器的主要套件及其相依關係；各套件的主要類別列於下表。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

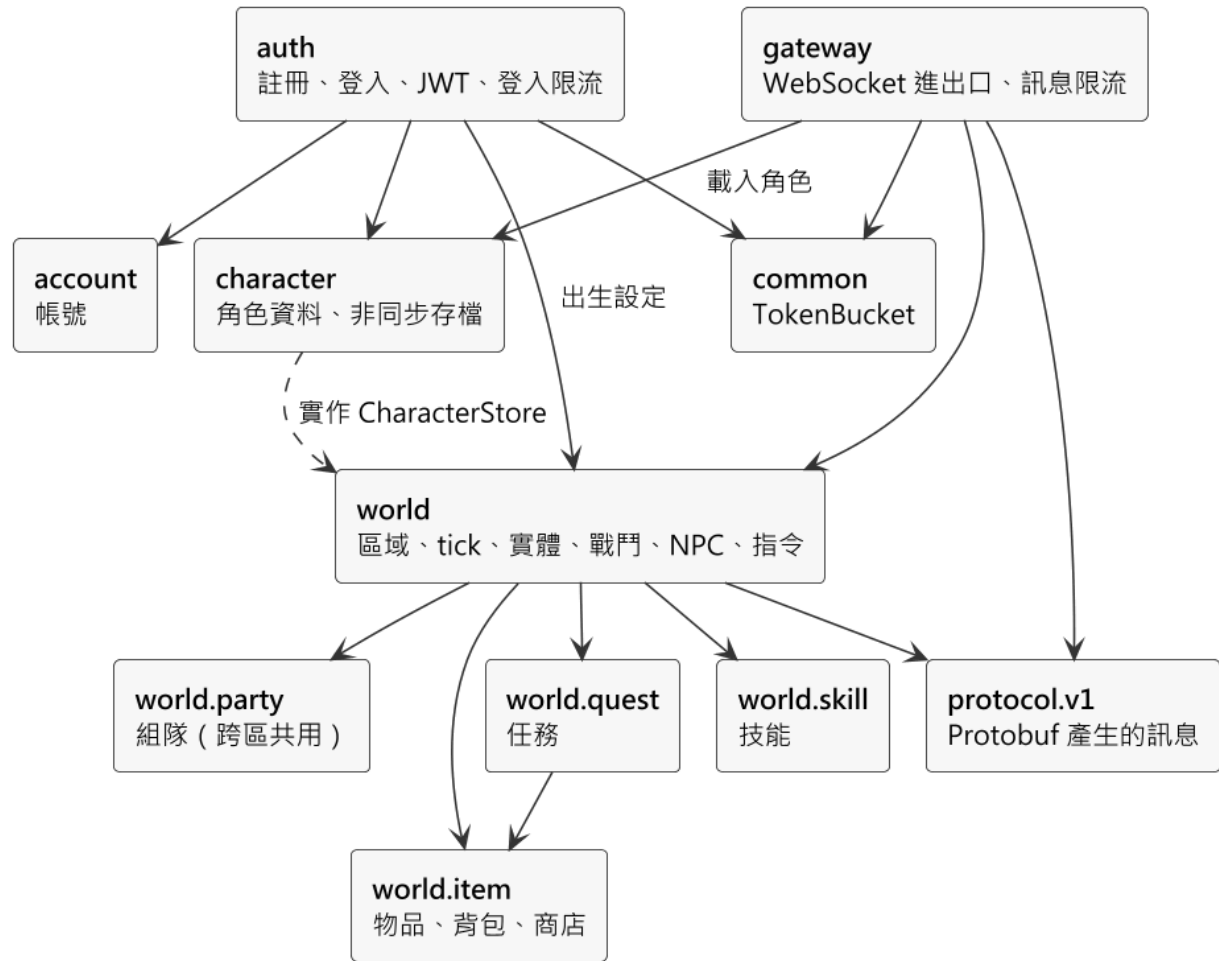


圖 7 伺服器套件相依關係

套件	職責	主要類別
auth	註冊、登入、JWT、登入頻率限制	AuthController、AuthService、TokenService、LoginThrottle、SecurityConfig
gateway	WebSocket 進出口、訊息頻率限制	GameWebSocketHandler、WebSocketPlayerConnection、ConnectionLimits、JwtHandshakeInterceptor
world	區域與 tick 迴圈、實體、戰鬥、NPC、指令、內容驗證	World、Zone、Entity、PlayerEntity、NpcEntity、Combat、NpcBrain、Commands、WorldContent、WorldProperties
world.item	物品、背包、裝備、掉寶、商店	Inventory、ItemTemplate、LootEntry、ShopDefinition
world.quest	任務定義與任務日誌	QuestDefinition、QuestLog
world.skill	技能定義、冷卻與增益	SkillDefinition、SkillBook
world.party	跨區域的組隊狀態	PartyService

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

character	角色資料與非同步存檔	PlayerCharacter、CharacterPersistence、各子表 Repository
account	帳號	Account、AccountRepository
common	共用元件	TokenBucket
protocol.v1	Protobuf 產生的訊息類別	ClientMessage、ServerMessage 等

5.3.1 區域事件迴圈 (Zone Tick)

每個區域以固定頻率（預設每秒 10 次）執行 tick，每個 tick 依序處理：

7. 取出佇列中的所有事件（進入、離開、移動、攻擊、文字指令），並移除連線已關閉的玩家。
8. NPC 行為：閒晃、主動攻擊、追太遠放棄回家、死亡後重生。
9. 戰鬥：追擊目標，到達攻擊距離後每秒出手一次。
10. 處理走向目的地的待辦動作：撿取物品、找商人或任務 NPC、走向出口。
11. 所有實體移動一步。
12. 定期工作：每 3 秒回復生命與內力、每秒回報隊員狀態、每 60 秒自動存檔、移除到期的地上物品。
13. 只廣播這個 tick 有變動的實體，並同步每位玩家有變動的數值、背包、任務與技能。

tick 中的例外會被捕捉並記錄，不會中斷後續的 tick。

5.3.2 通訊協定 (Protocol)

瀏覽器與伺服器之間的即時訊息以 Protobuf 編碼，透過 WebSocket 傳送；登入與註冊則使用 REST（JSON）。用戶端只有三種訊息：MoveTo（點擊移動）、Attack（點擊攻擊）與 Command（文字指令）。其餘所有操作（撿取、裝備、交易、組隊、任務、技能）都以文字指令表達，伺服器只有一條指令處理路徑。

伺服器訊息	用途
Welcome	進入區域（包括換區），客戶端據此重建場景
WorldSnapshot/EntityLeft	實體狀態的完整快照或增量更新；實體離開
TextOutput	MUD 風格的文字訊息（系統、說話、房間、隊伍頻道）
CombatEvent	一次攻擊或治療的結果，用於顯示傷害數字
SelfStats	自己的等級、生命、內力、經驗、銅錢與攻防
Inventory	完整背包內容
ShopView/ShopClosed	商店的販賣清單與收購報價；離開商人時關閉
PartyView	隊員的等級、生命與所在區域
QuestLog/QuestDialog/QuestMarkers	任務日誌、NPC 任務對話、NPC 頭上的「!」「?」標記
SkillBook	已學會的技能與冷卻、附近訓練師可教的技能

5.3.3 遊戲內容與驗證 (Content)

遊戲內容寫在 content.yml，與程式設定分開，由 Spring 綁定成不可變的記錄物件，再組成 WorldContent 交給各區域。伺服器啟動時會檢查所有引用是否存在（例如區域引用的 NPC、掉寶表與商店貨品引用的物

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

品、任務與技能引用的 NPC），也檢查發任務與教技能的 NPC 必須是不能攻擊的友善 NPC 或商人；有任何錯誤就拒絕啟動。目前的內容數量如下：

內容	數量	說明
區域	2	新手村、黑松林
NPC 模板	9	怪物 6 種、商人 2 位、村長 1 位
物品	22	裝備、消耗品、材料
商店	2	陳記雜貨、獵人小屋
任務	8	從村長到獵人老李的任務鏈
技能	5	單體、範圍、治療、增益

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

5.4 資料庫綱要 (Database Schema)

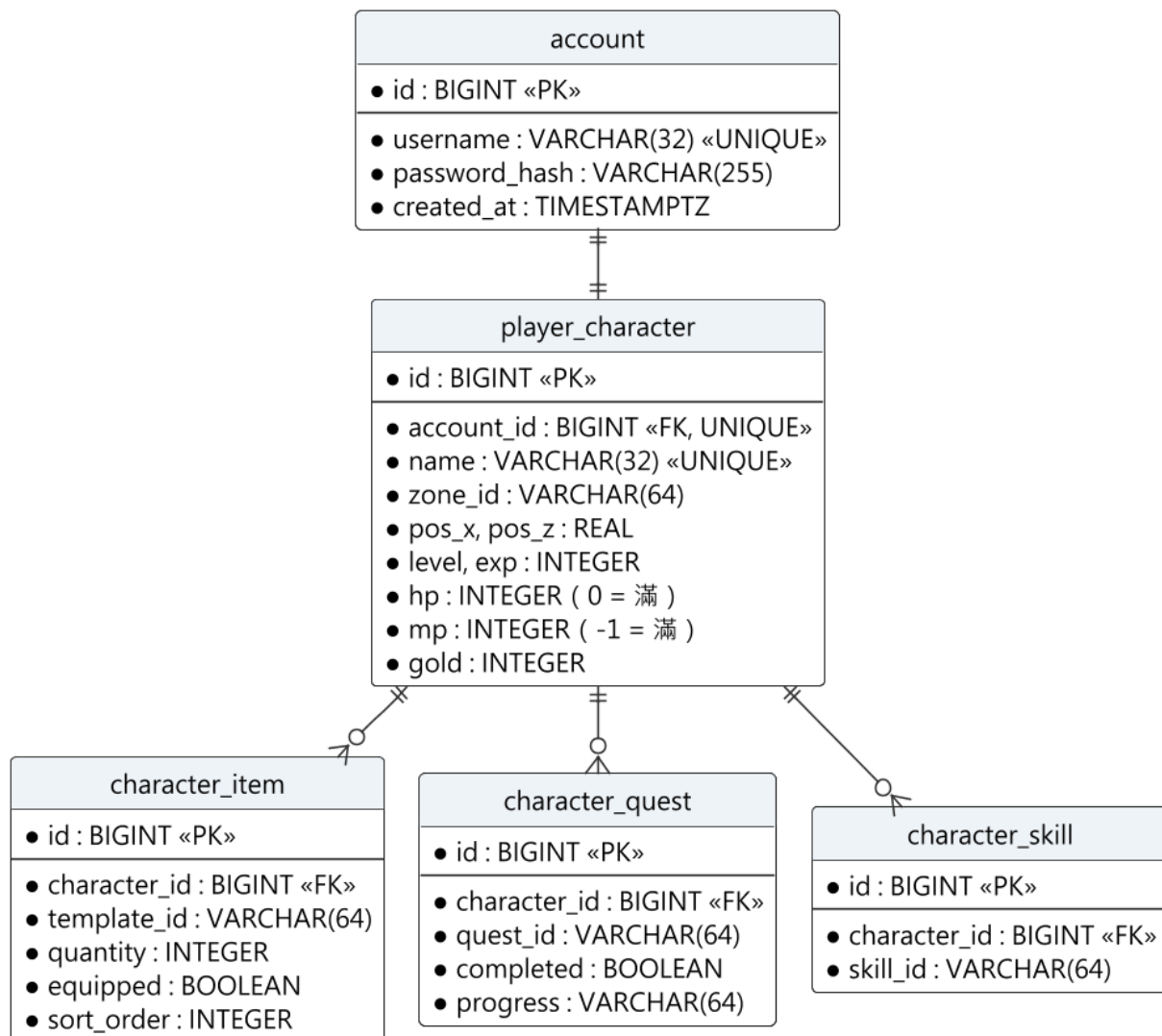


圖 8 資料庫綱要

資料庫結構由 Flyway 管理 (V1~V6)。每個帳號對應一個同名角色；角色的背包、任務進度與學會的技能存在三張子表中，每次存檔都在同一個交易內整份覆寫（背包最多 20 筆），以維持一致性。生命欄位為 0、內力欄位為 -1 時代表「滿」，用於新角色與舊資料。技能冷卻、增益效果與隊伍只存在記憶體中，不寫入資料庫。

6. 程序視圖 (Process View)

程序視圖描述系統分成哪些執行緒與程序、它們如何溝通。本系統的關鍵在於：遊戲狀態由區域執行緒獨占，其他執行緒只能透過佇列與區域溝通，因此遊戲邏輯不需要任何鎖。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

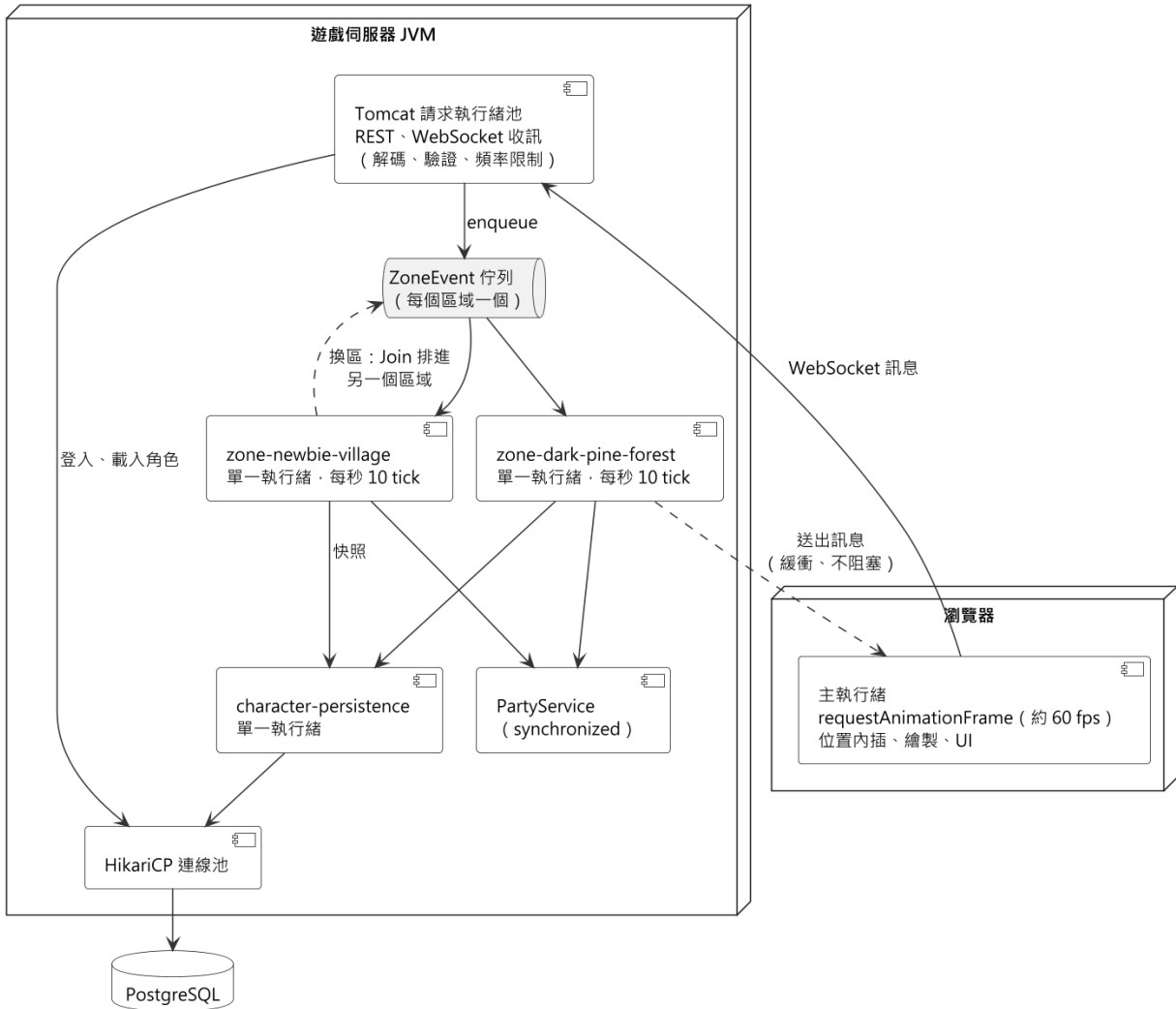


圖 9 執行緒與程序

6.1 瀏覽器主執行緒

以 requestAnimationFrame 驅動繪製（約每秒 60 格），在格與格之間內插實體位置；收到 WebSocket 訊息時更新場景與介面。

6.2 Tomcat 請求執行緒

處理 REST 請求，以及 WebSocket 訊息的接收、解碼、驗證與頻率限制，最後把事件排入區域的佇列。這些執行緒會存取資料庫（登入、進場時載入角色），但從不存取區域的狀態。

6.3 區域執行緒

每個區域一條執行緒（例如 zone-newbie-village、zone-dark-pine-forest），以固定頻率執行 tick。區域的所有狀態都只由這條執行緒讀寫。送出訊息時透過 ConcurrentWebSocketSessionDecorator 緩衝，不會阻塞 tick；客戶端太慢導致緩衝超過 512 KB 或送出超過 2 秒時，直接中斷該連線。

6.4 存檔執行緒

character-persistence 是單一執行緒，依序把角色快照寫入資料庫，確保同一角色的存檔順序正確，也讓區域

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

tick 不必等待資料庫。

6.5 共用狀態：PartyService

隊員可能分散在不同區域，所以組隊狀態是全域共用、以 `synchronized` 保護的物件。區域只取得不可變的隊伍快照；要送給隊員的訊息先收集起來，離開鎖之後才送出，避免一條慢連線拖住其他區域。

6.6 執行緒之間的規則

- 跨執行緒只能透過 `ZoneEvent` 佇列進入區域；區域之間交接玩家也是排入對方的佇列。
- 換區時先排入 `Join` 再切換連線路由；換區途中斷線時，離開事件可能送到舊區域，因此每個區域每個 tick 都會移除連線已關閉的玩家，不會留下殘影。
- 關機順序：World 停止所有區域，並把區域內所有玩家存檔；存檔執行緒最多等待 10 秒寫完；最後才關閉資料庫連線池。

7. 部署視圖 (Deployment View)

部署視圖描述軟體在哪些實體節點上執行，以及節點之間如何連接。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

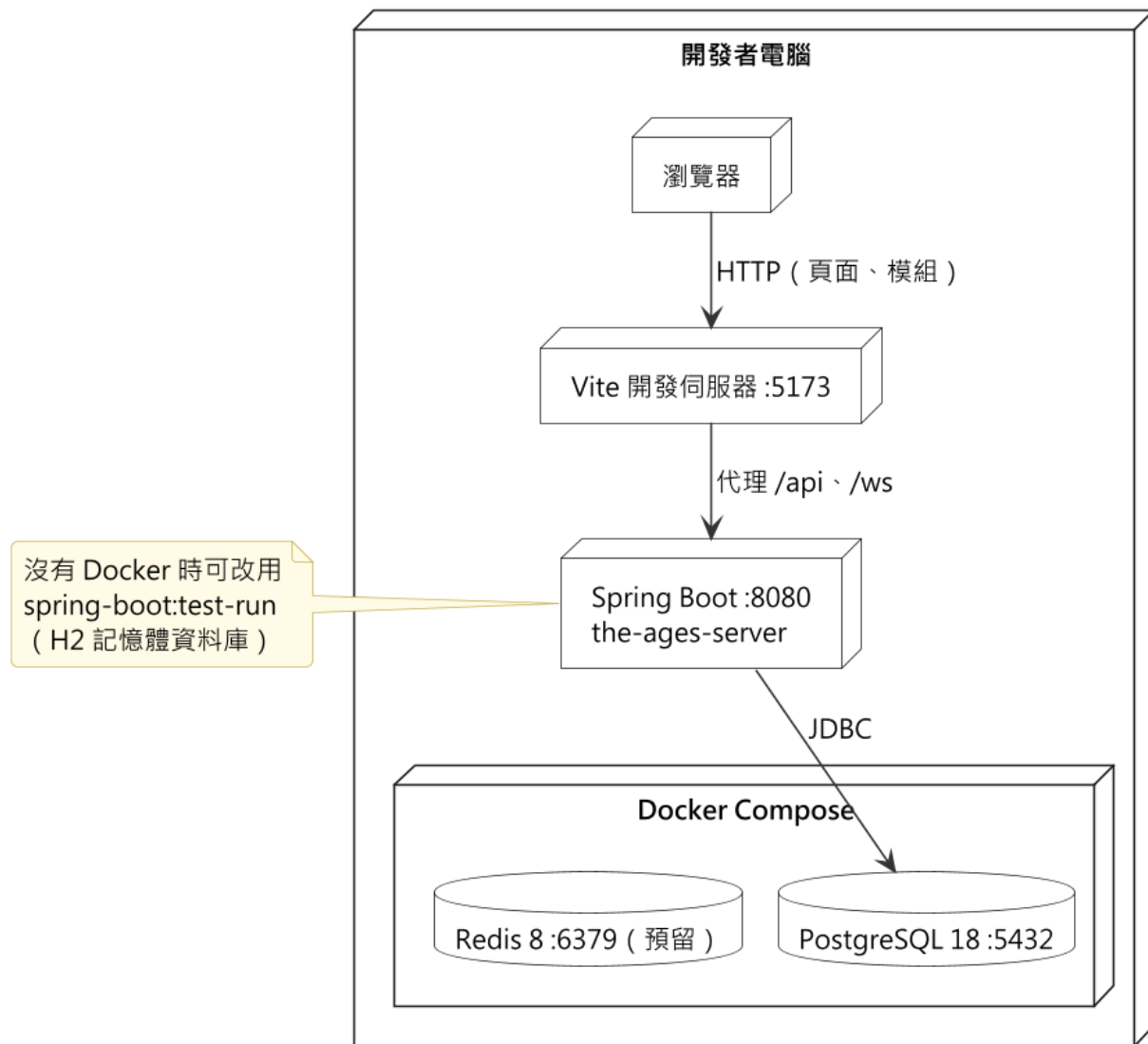


圖 10 開發環境

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

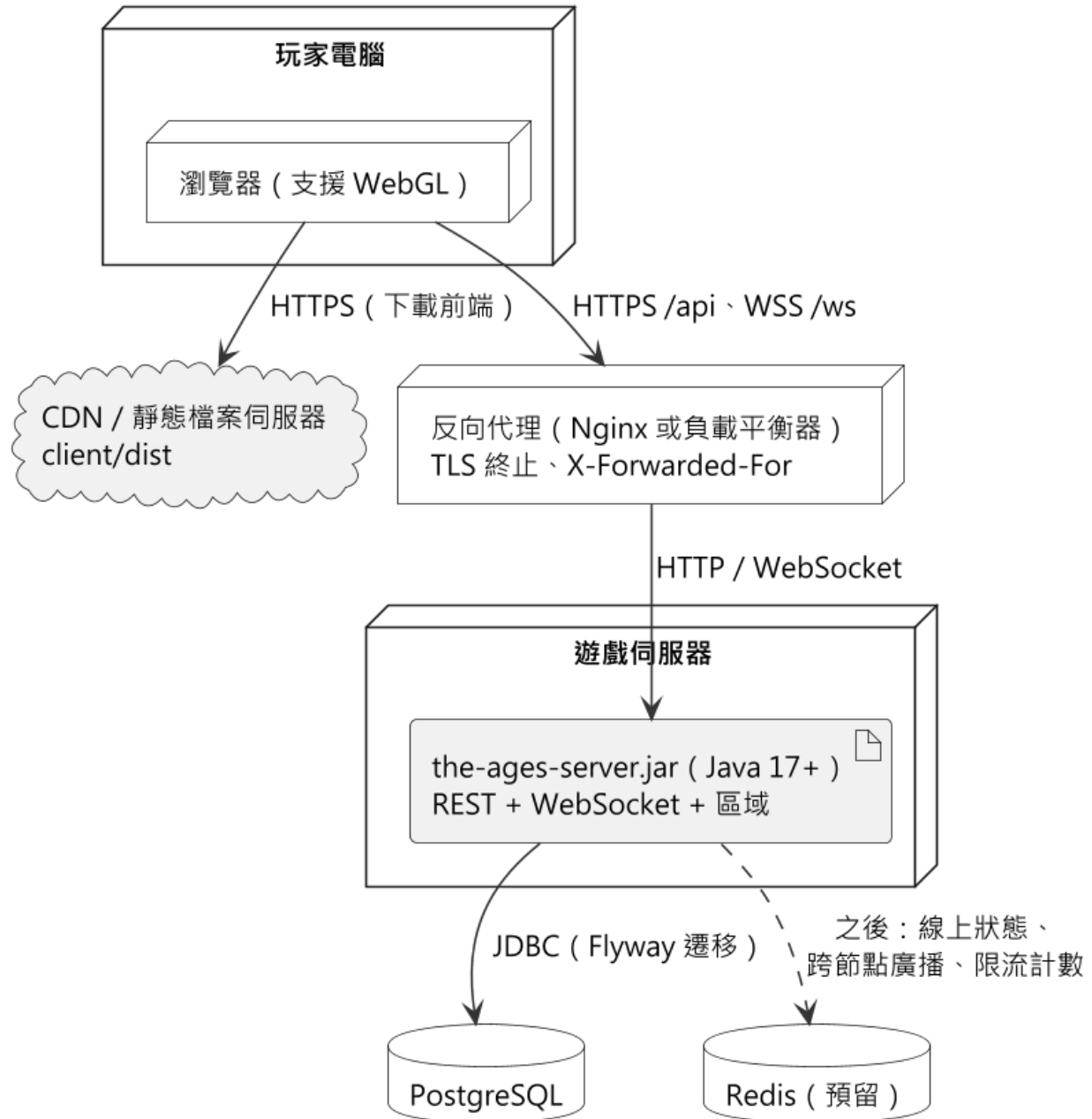


圖 11 規劃的正式環境

7.1 用戶端 (瀏覽器)

任何支援 WebGL 與 WebSocket 的現代瀏覽器，不需安裝。前端以 HTTPS 下載，遊戲連線使用 WSS。

7.2 靜態檔案伺服器 / CDN

提供 Vite 打包後的前端靜態檔 (client/dist)。前端與伺服器分離部署，可以獨立更新與擴充。

7.3 反向代理

負責 TLS 終止，並把 /api 與 /ws 轉給遊戲伺服器。部署在反向代理之後時，必須設定 server.forward-

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

headers-strategy，並且只信任自己代理送來的 X-Forwarded-For；否則所有玩家會共用代理的 IP 額度，攻擊者也可能偽造這個標頭繞過登入頻率限制。theages.websocket.allowed-origins 需設定為正式的前端網址。

7.4 遊戲伺服器

單一 JVM（Java 17 以上）執行 the-ages-server.jar，同時提供 REST、WebSocket 與所有區域的 tick 迴圈（預設連接埠 8080）。JWT 金鑰必須以環境變數 THEAGES_JWT_SECRET 覆寫開發用的預設值。伺服器支援優雅關機，關機前會把所有線上玩家存檔。

7.5 資料庫伺服器

PostgreSQL。伺服器啟動時由 Flyway 自動套用資料庫遷移。

7.6 快取伺服器（預留）

Redis 已列在 docker-compose.yml 中但尚未使用。之後擴充成多台遊戲伺服器時，用於線上狀態、跨節點廣播與共用的限流計數。

7.7 開發環境

Vite 開發伺服器（連接埠 5173）提供前端，並把 /api 與 /ws 代理到 Spring Boot（連接埠 8080）；PostgreSQL 與 Redis 以 Docker Compose 啟動。沒有 Docker 時，可以用 test profile 改以 H2 記憶體資料庫啟動伺服器。

8. 實作視圖 (Implementation View)

實作視圖描述原始碼與建置產物的組織方式，以及它們和邏輯分層的對應關係。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

8.1 概觀 (Overview)

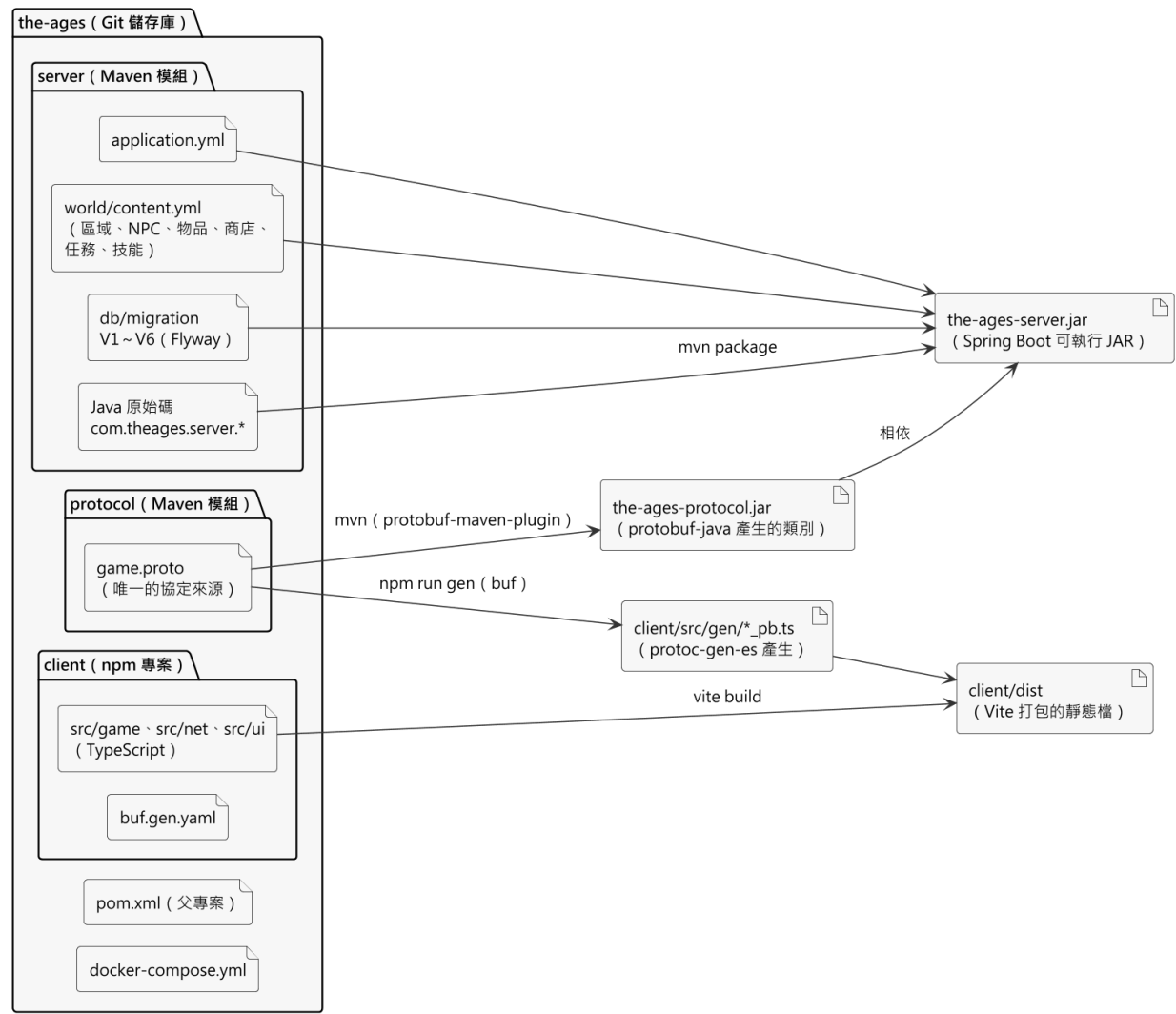


圖 12 原始碼模組與建置產物

目錄	內容
pom.xml	Maven 父專案（Spring Boot 4.1.1、Java 17），包含 protocol 與 server 兩個模組
protocol/	game.proto；建置時由 protobuf-maven-plugin 產生 Java 類別
server/	遊戲伺服器原始碼、設定檔、遊戲內容與資料庫遷移
client/	前端 TypeScript 原始碼；npm run gen 以 buf 從同一份 game.proto 產生 TypeScript
docker-compose.yml	開發用的 PostgreSQL 與 Redis

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

8.2 層級 (Layers)

邏輯層	實作位置
展示層	client/src/game 、 client/src/ui 、 client/src/net
閘道層	com.theages.server.auth 、 com.theages.server.gateway
遊戲核心層	com.theages.server.world 及其子套件 item 、 quest 、 skill 、 party
持久層	com.theages.server.character 、 com.theages.server.account 、 db/migration
共用	com.theages.server.common 、 protocol 模組

8.3 部署元件 (Deployment Components)

- the-ages-server.jar：Spring Boot 可執行 JAR，內含 the-ages-protocol.jar、application.yml、content.yml 與資料庫遷移檔。
- client/dist：Vite 打包的前端靜態檔（HTML、JavaScript、CSS）。

8.4 建置與測試

- mvn verify：建置伺服器並執行 110 個測試，包含遊戲核心的單元測試，以及以 H2 啟動真正伺服器的整合測試（註冊、登入、WebSocket 進場、頻率限制）。
- npm run build：產生協定程式碼、TypeScript 型別檢查並打包前端。

9. 規模與效能 (Size and Performance)

本系統尚未進行負載測試，以下為架構上支援規模與效能的設計，實際容量需在正式環境實測後確認。

- 區域每秒 10 次 tick，每個 tick 有 100 毫秒的時間預算；tick 的工作量與區域內的實體數量成正比。增加區域即增加執行緒，可以利用多核心。
- 只在進場時送完整快照，之後每個 tick 只廣播有變動的實體；訊息以 Protobuf 二進位編碼，體積小。
- 前端以指數平滑把伺服器 10 Hz 的位置更新補成約 60 fps 的動畫，不需要提高網路更新頻率。
- 存檔不佔用 tick：離線、換區、每 60 秒自動存檔時，交給存檔執行緒非同步寫入。
- 頻率限制保護伺服器：每條連線的點擊移動最多連續 10 則、平均每秒 5 則；點擊攻擊連續 5 則、每秒 2 則；文字指令連續 8 則、每秒 2 則；單則訊息上限 4 KB；10 秒內被擋 50 次即斷線。
- 登入保護：每個 IP 每分鐘 10 次登入嘗試、每 2 分鐘 1 次註冊；同一帳號 15 分鐘內失敗 5 次即鎖定 15 分鐘。

10. 品質 (Quality)

軟體架構以下列方式支援各項品質需求：

10.1 安全性

- 伺服器擁有最終決定權，客戶端無法自行決定位置、傷害或交易結果。
- 密碼以 bcrypt 雜湊；JWT 以 HS256 簽章，有效期 12 小時，WebSocket 握手時驗證。
- WebSocket 只接受設定中允許的來源網址。
- 遊戲訊息與登入都有頻率限制；帳號鎖定與一致的回應時間避免帳號被試探。

時空之門 The Ages	Version: 1.0
軟體架構文件	Date: 2026/10/06
The Ages 專案	

10.2 可靠性

- 角色、背包、任務與技能在同一個交易內存檔；每 60 秒自動存檔，離線、換區與關機時也會存檔。
- tick 中的例外會被捕捉並記錄，不會讓區域停止。
- 斷線的玩家會在下一個 tick 被移除；同一角色重複登入時沿用記憶體中的狀態，舊連線會被踢掉。

10.3 可維護性

- 遊戲核心不依賴 Spring，可以完全以單元測試驗證；目前共有 110 個自動化測試。
- 遊戲內容與程式分離，企劃修改 content.yml 不需要改程式；錯誤的內容在啟動時就會被發現。
- 協定只有一份來源，前後端的訊息定義不會不一致。

10.4 已知限制

- 所有資料庫遷移（V1～V6）目前只在 H2 上驗證過，尚未在 PostgreSQL 上實際執行。
- 技能冷卻、增益效果與隊伍只存在記憶體中，換區、重新登入或伺服器重啟後會重置。
- 登入頻率限制只存在單台伺服器的記憶體中，擴充成多台時需改用共用儲存。
- 玩家斷線後立刻重連，可能讀到尚未寫入的舊位置（存檔為非同步）。
- 尚未支援玩家對戰（PvP）與玩家之間的交易；3D 場景目前為程式產生的佔位模型。